DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Índice general

Resumen I

1. Introducción general 1
- 1.1. Domótica en motorhomes . . . 1
- 1.2. Motivaciones del trabajo . . . 1
- 1.3. Estado del arte . . . 2
- 1.4. Objetivos y alcances . . . 4

2. Introducción específica 7
- 2.1. Protocolos de comunicación . . . 7
  - 2.1.1. Protocolo CAN . . . 7
  - 2.1.2. Protocolo Modbus TCP . . . 8
  - 2.1.3. Solicitudes HTTP . . . 8
- 2.2. Componentes de hardware . . . 9
  - 2.2.1. Microcontrolador ESP32 . . . 9
  - 2.2.2. Módulo transceiver . . . 9
  - 2.2.3. Módulo Ethernet W5100 . . . 10
- 2.3. Herramientas de software . . . 11
  - 2.3.1. Entorno de desarrollo Arduino . . . 11
  - 2.3.2. Funcionamiento del ESP32 . . . 11
  - 2.3.3. Desarrollo de la aplicación móvil . . . 12

3. Diseño e implementación 13
- 3.1. Desarrollo de hardware . . . 13
  - 3.1.1. Comunicación con la red Enertik . . . 14
  - 3.1.2. Comunicación con la red Victron . . . 16
  - 3.1.3. Prototipo experimental . . . 17
  - 3.1.4. Desarrollo de PCB final y listado de componentes . . . 18
- 3.2. Desarrollo de firmware . . . 20
  - 3.2.1. Tarea: recepción UART . . . 21
  - 3.2.2. Tarea: decodificación de comando CAN . . . 21
  - 3.2.3. Tarea: comunicación Modbus TCP . . . 22
  - 3.2.4. Tarea: pedido de datos . . . 23
  - 3.2.5. Tarea: procesamiento de comandos recibidos . . . 23
  - 3.2.6. Tarea: control de servidor . . . 24
- 3.3. Desarrollo de la aplicación . . . 25
  - 3.3.1. Aplicación: pestaña principal . . . 25
  - 3.3.2. Aplicación: pestaña luces . . . 27
  - 3.3.3. Aplicación: pestaña motores . . . 27
  - 3.3.4. Aplicación: pestaña energía . . . 28
  - 3.3.5. Aplicación: generación de instalador para Android . . . 29

4. Ensayos y resultados 31

V

Índice general

Resumen I

1. Introducción general 1
- 1.1. Domótica en motorhomes . . . 1
- 1.2. Motivaciones del trabajo . . . 1
- 1.3. Estado del arte . . . 2
- 1.4. Objetivos y alcances . . . 4

2. Introducción específica 7
- 2.1. Protocolos de comunicación . . . 7
  - 2.1.1. Protocolo CAN . . . 7
  - 2.1.2. Protocolo Modbus TCP . . . 8
  - 2.1.3. Solicitudes HTTP . . . 8
- 2.2. Componentes de hardware . . . 9
  - 2.2.1. Microcontrolador ESP32 . . . 9
  - 2.2.2. Módulo transceiver . . . 9
  - 2.2.3. Módulo Ethernet W5100 . . . 10
- 2.3. Herramientas de software . . . 11
  - 2.3.1. Entorno de desarrollo Arduino . . . 11
  - 2.3.2. Funcionamiento del ESP32 . . . 11
  - 2.3.3. Desarrollo de la aplicación móvil . . . 12

3. Diseño e implementación 15
- 3.1. Análisis del software . . . 15

4. Ensayos y resultados 17
- 4.1. Pruebas funcionales del hardware . . . 17

5. Conclusiones 19
- 5.1. Conclusiones generales . . . 19
- 5.2. Próximos pasos . . . 19

Bibliografía 21

4.1. Pruebas funcionales del hardware 31

5. Conclusiones **33**
5.1. Conclusiones generales 33
5.2. Próximos pasos 33

Bibliografía **35**

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control. 3
1.2. Arquitectura de comunicación del sistema con el ESP32 como intermediario entre PRC10, Cerbo GX y la aplicación móvil. 5

2.1. Microcontrolador ESP32[1]. 9
2.2. Funcionamiento del módulo transceiver. 10
2.3. Módulo W5100[2]. 10
2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio. 12

3.1. Trayecto que sigue el ESP32 para llegar a cada uno de los módulos. 13
3.2. Módulo transceiver comercial perteneciente a Enertik. 14
3.3. Diagrama de como se estable la comunicación CAN entre la red del motorhome y el ESP32. 15
3.4. Diseño de divisor resistivo para adaptación UART. 15
3.5. Prototipo experimental del equipo. 17
3.6. Diseño final para PCB del equipo. 19
3.7. Relación entre las distintas tareas del firmware y su comunicación con el exterior. 21
3.8. Pestaña *home* de la aplicación. 26
3.9. Pestaña luces de la aplicación. 27
3.10. Pestaña motores de la aplicación. 28
3.11. Pestaña energía de la aplicación. 29

VII

Índice de figuras

1.1. Arquitectura centralizada de la red de domótica con el módulo PRC10 como unidad de control. 3
1.2. Arquitectura de comunicación del sistema con el ESP32 como intermediario entre PRC10, Cerbo GX y la aplicación móvil. 5

2.1. Microcontrolador ESP32. 9
2.2. Funcionamiento del módulo transceiver. 10
2.3. Módulo W5100. 10
2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio. 12

Índice de tablas

3.1. Componentes utilizados en el PCB final. 20

Índice de tablas

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación empleados en el desarrollo del trabajo, los componentes de hardware empleados para cumplir con los objetivos planteados y las herramientas utilizadas para el desarrollo del software.

2.1. Protocolos de comunicación

Como se mencionó en la sección 1.4, el dispositivo debía cumplir con la funcionalidad de actuar como intermediario entre las redes establecidas por Enertik y Victron, así como también constituir un punto de acceso para que la aplicación móvil pueda operar sobre el sistema.

Para lograr este objetivo, fue necesario implementar distintos protocolos de comunicación que permitan el intercambio de información entre los subsistemas involucrados, incluyendo CAN, Modbus TCP y una interfaz basada en HTTP (*HyperText Transfer Protocol*) [9].

2.1.1. Protocolo CAN

El protocolo CAN es un protocolo serie ampliamente utilizado en la industria automotriz [6]. Permite la comunicación entre múltiples dispositivos sin la necesidad de un controlador central y proporciona un intercambio de datos rápido y confiable.

En este protocolo, la transmisión de datos se realiza a través de un par de conductores denominado CAN H y CAN L, en donde se emplea una señalización diferencial. Esta técnica consiste en transmitir la información como la diferencia de tensión entre ambos conductores, lo que proporciona una alta inmunidad a interferencias electromagnéticas, ya que el ruido externo tiende a afectar a ambos cables de manera similar y, por lo tanto, es cancelado al evaluar la diferencia entre ellos.

Por otra parte, la comunicación en una red CAN se basa en un esquema de difusión, donde todos los nodos reciben los mensajes transmitidos. Cada mensaje incluye un identificador que permite a los dispositivos determinar si la información es relevante para ellos, ignorando aquellos mensajes que no corresponden a su funcionamiento.

7

Capítulo 2

Introducción específica

En este capítulo se describen los distintos protocolos de comunicación empleados en el desarrollo del trabajo, los componentes de hardware implementados para cumplir con los objetivos planteados y las herramientas utilizadas para el desarrollo del software.

2.1. Protocolos de comunicación

Como se mencionó en la sección 1.4, el dispositivo debía cumplir con la funcionalidad de actuar como intermediario entre las redes establecidas por Enertik y Victron, así como también constituir un punto de acceso para que la aplicación móvil pueda operar sobre el sistema.

Para lograr este objetivo, fue necesario implementar distintos protocolos de comunicación que permitan el intercambio de información entre los subsistemas involucrados, incluyendo CAN, Modbus TCP y una interfaz basada en HTTP.

2.1.1. Protocolo CAN

El protocolo CAN es un protocolo serie ampliamente utilizado en la industria automotriz. Permite la comunicación entre múltiples dispositivos sin la necesidad de un controlador central y proporciona un intercambio de datos rápido y confiable.

En este protocolo, la transmisión de datos se realiza a través de un par de conductores denominado CAN H y CAN L, en donde se emplea una señalización diferencial. Esta técnica consiste en transmitir la información como la diferencia de tensión entre ambos conductores, lo que proporciona una alta inmunidad a interferencias electromagnéticas, ya que el ruido externo tiende a afectar a ambos cables de manera similar y, por lo tanto, es cancelado al evaluar la diferencia entre ellos.

En cuanto a la representación lógica, un bit dominante (0 lógico) se caracteriza por una diferencia de tensión significativa entre CAN H y CAN L (normalmente un conductor adquiere un valor de tensión igual a la fuente y el otro a GND), mientras que un bit recesivo (1 lógico) se corresponde con un estado en el que ambos conductores presentan niveles de tensión similares.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

2.1.2. Protocolo Modbus TCP

El protocolo Modbus es uno de los protocolos más utilizados en la industria de la automatización. Su función principal es permitir que distintos dispositivos electrónicos se comuniquen entre sí de una forma sencilla y entandarizada.

En una red Modbus, exite una relación maestro-esclavo entre los dispositivos. El dispositivo maestro es el encargado de iniciar una comunicación y solicitar información. El esclavo se encarga de atender estas solicitudes y entregar los datos que el maestro requiera.

En el caso de Modbus TCP, este protocolo se implementa sobre redes Ethernet, donde emplea el protocolo TCP/IP [10] como medio de transporte. Esto permite que los dispositivos puedan comunicarse a través de redes locales, identificándose mediante direcciones IP.

La comunicación se basa en un esquema de tipo *request-response*, donde el cliente (maestro) envía una solicitud a un servidor (esclavo), y este último procesa la petición y devuelve una respuesta [8]. Dentro de estas solicitudes se incluyen funciones específicas que permiten leer o escribir distintos tipos de registros, tales como *holding registers* o *input registers*.

En el contexto de este trabajo, el protocolo Modbus TCP es utilizado para establecer la comunicación con el módulo Cerbo GX, que expone diferentes registros que pueden ser consultados y controlados de forma remota.

2.1.3. Solicitudes HTTP

El protocolo HTTP es un protocolo de aplicación ampliamente utilizado para la comunicación entre clientes y servidores. Su funcionamiento se basa en un modelo de tipo *request-response*, donde un cliente envía una solicitud (request) a un servidor, este procesa la petición y devuelve una respuesta (response) con la información requerida.

Las solicitudes HTTP permiten a los clientes acceder a recursos, enviar información y ejecutar acciones sobre un servidor. Estas peticiones están compuestas por distintos elementos, tales como el método de la solicitud, la dirección del recurso solicitado y, en algunos casos, un cuerpo de datos que contiene la información a enviar.

Existen diferentes tipos de métodos HTTP, cada uno con una función específica. Entre los más utilizados se encuentran el método GET, empleado para solicitar información al servidor, y el método POST, utilizado para enviar datos y generar cambios en el estado del sistema. Otros métodos relevantes incluyen PUT, orientado a la actualización de recursos, y DELETE, destinado a su eliminación [11].

En el contexto de este trabajo, las solicitudes HTTP son utilizadas como mecanismo de comunicación entre la aplicación móvil y el dispositivo basado en ESP32, lo que permite el envío de comandos de control y la obtención de información del sistema.

Por otra parte, la comunicación en una red CAN se basa en un esquema de difusión, donde todos los nodos reciben los mensajes transmitidos. Cada mensaje incluye un identificador que permite a los dispositivos determinar si la información es relevante para ellos, ignorando aquellos mensajes que no corresponden a su funcionamiento.

2.1.2. Protocolo Modbus TCP

El protocolo Modbus es uno de los protocolos mas utilizados en la industria de la automatización. Su función principal es permitir que distintos dispositivos electrónicos se comuniquen entre sí de una forma sencilla y entandarizada.

En una red Modbus, exite una relación maestro-esclavo entre los dispositivos. El dispositivo maestro es el encargado de iniciar una comunicación y solicitar información. El esclavo se encarga de atender estas solicitudes y entregar los datos que el maestro requiera.

En el caso de Modbus TCP, este protocolo se implementa sobre redes Ethernet, donde emplea el protocolo TCP/IP [9] como medio de transporte. Esto permite que los dispositivos puedan comunicarse a través de redes locales, identificándose mediante direcciones IP.

La comunicación se basa en un esquema de tipo *request-response*, donde el cliente (maestro) envía una solicitud a un servidor (esclavo), y este último procesa la petición y devuelve una respuesta. Dentro de estas solicitudes se incluyen funciones específicas que permiten leer o escribir distintos tipos de registros, tales como *holding registers* o *input registers*.

En el contexto de este trabajo, el protocolo Modbus TCP es utilizado para establecer la comunicación con el módulo Cerbo GX, que expone diferentes registros que pueden ser consultados y controlados de forma remota.

2.1.3. Solicitudes HTTP

El protocolo HTTP (*HyperText Transfer Protocol*) [10] es un protocolo de aplicación ampliamente utilizado para la comunicación entre clientes y servidores. Su funcionamiento se basa en un modelo de tipo *request-response*, donde un cliente envía una solicitud (request) a un servidor, este procesa la petición y devuelve una respuesta (response) con la información requerida.

Las solicitudes HTTP permiten a los clientes acceder a recursos, enviar información y ejecutar acciones sobre un servidor. Estas peticiones están compuestas por distintos elementos, tales como el método de la solicitud, la dirección del recurso solicitado y, en algunos casos, un cuerpo de datos que contiene la información a enviar.

Existen diferentes tipos de métodos HTTP, cada uno con una función específica. Entre los más utilizados se encuentran el método GET, empleado para solicitar información al servidor, y el método POST, utilizado para enviar datos y generar cambios en el estado del sistema. Otros métodos relevantes incluyen PUT, orientado a la actualización de recursos, y DELETE, destinado a su eliminación.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

2.2. Componentes de hardware

Para el desarrollo de hardware fue necesaria la utilización de ciertos componentes. El principal de ellos fue el ESP32 como microcontrolador del equipo. A su vez, para poder adaptarlo a los distintos protocolos de comunicación, fue necesario el agregado de dos periféricos adicionales: el módulo W5110 y el MCP2551.

2.2.1. Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por la empresa Espressif Systems. Se trata de un dispositivo SoC (*System on Chip*) que integra múltiples periféricos y capacidades de procesamiento en un único dispositivo.

Entre sus principales características, el ESP32 dispone de puertos de propósito general (GPIO), conversores analógico-digitales (ADC), módulos de comunicación serie como UART, SPI e I^2C, temporizadores y periféricos para generación de señales PWM.

Además, el dispositivo incorpora conectividad inalámbrica, incluyendo Wi-Fi y Bluetooth, lo que lo convierte en una plataforma adecuada para el desarrollo de aplicaciones orientadas al campo IoT (*Internet of Things*). Todas las características mencionadas anteriormente vuelven al ESP32 en una opción mas que adecuada para el desarrolo del trabajo

FIGURA 2.1. Microcontrolador ESP32[1].

2.2.2. Módulo transceiver

Todos los dispositivos que pertenecen a la red de domótica de Enertik se conectan a través de un cableado de cuatro conductores (+12 V, GND, CAN H y CAN L). Este esquema permite tanto la alimentación como la comunicación de los equipos conectados a la red de manera sencilla.

Por otra parte, el protocolo de comunicación empleado corresponde a una biblioteca propia desarrollada por Enertik, basada en el estándar CAN. Para adaptar el ESP32 a esta red y permitir su correcta integración, fue necesaria la implementación de un módulo transceiver.

[1]Imagen tomada de https://robu.in/product/esp32-38pin-development-board-wifibluetooth-ultra-low-power-consumption-dual-core/.

En el contexto de este trabajo, las solicitudes HTTP son utilizadas como mecanismo de comunicación entre la aplicación móvil y el dispositivo basado en ESP32, lo que permite el envío de comandos de control y la obtención de información del sistema.

2.2. Componentes de hardware

Para el desarrollo de hardware fue necesaria la utilización de ciertos componentes. El principal de ellos fue el ESP32 como microcontrolador del equipo. A su vez, para poder adaptarlo a los distintos protocolos de comunicación, fue necesario el agregado de dos periféricos adicionales: el módulo W5110 y el MCP2551.

2.2.1. Microcontrolador ESP32

El ESP32 es un microcontrolador desarrollado por la empresa Espressif Systems. Se trata de un dispositivo SoC (*System on Chip*) que integra múltiples periféricos y capacidades de procesamiento en un único dispositivo.

Entre sus principales características, el ESP32 dispone de puertos de propósito general (GPIO), conversores analógico-digitales (ADC), módulos de comunicación serie como UART, SPI e I^2C, temporizadores y periféricos para generación de señales PWM.

Además, el dispositivo incorpora conectividad inalámbrica, incluyendo Wi-Fi y Bluetooth, lo que lo convierte en una plataforma adecuada para el desarrollo de aplicaciones orientadas al campo IoT (*Internet of Things*). Todas las características mencionadas anteriormente vuelven al ESP32 en una opción mas que adecuada para el desarrolo del trabajo

FIGURA 2.1. Microcontrolador ESP32.

2.2.2. Módulo transceiver

Todos los dispositivos que pertenecen a la red de domótica de Enertik se conectan a través de un cableado de cuatro conductores (+12 V, GND, CAN H y CAN L).

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Este módulo actúa como interfaz entre el ESP32 y la red de domótica, y está compuesto por un microcontrolador PIC [12], en el que se implementa la biblioteca de comunicación, y un transceptor CAN MCP2551 [13], encargado de la adaptación de niveles eléctricos para la transmisión diferencial sobre el bus.

Como se puede apreciar en la figura 2.2, este módulo realiza la conversión de la red CAN, proveniente del sistema de domótica, a una interfaz de comunicación serial UART (*Universal Asynchronous Receiver-Transmitter*) [14], compatible con el ESP32.

Esta solución permite desacoplar la gestión del protocolo de comunicación del procesamiento principal del sistema, lo que simplifica el desarrollo del firmware y facilita la integración del dispositivo en la red de domótica existente.

FIGURA 2.2. Funcionamiento del módulo transceiver.

2.2.3. Módulo Ethernet W5100

Para lograr la comunicación mediante el protocolo Modbus TCP, fue necesaria la utilización de un módulo Ethernet basado en el circuito W5100 [15].

El W5100 es un controlador Ethernet que permite dotar de conectividad de red a sistemas embebidos, gestionando internamente la pila de protocolos TCP/IP. La comunicación con el microcontrolador se realiza a través de una interfaz SPI (*Serial Peripheral Interface*)[16] , lo que facilita su integración en plataformas como el ESP32.

Mediante el uso de este módulo, el dispositivo desarrollado puede establecer conexiones sobre redes Ethernet, lo que permite la implementación del protocolo Modbus TCP para la comunicación con equipos del ecosistema Victron.

FIGURA 2.3. Módulo W5100[2].

[2]Imagen tomada de https://grobotronics.com/wiznet-w5100-ethernet-network-module.html

Este esquema permite tanto la alimentación como la comunicación de los equipos conectados a la red de manera sencilla.

Por otra parte, el protocolo de comunicación empleado corresponde a una biblioteca propia desarrollada por Enertik, basada en el estándar CAN. Para adaptar el ESP32 a esta red y permitir su correcta integración, fue necesaria la implementación de un módulo transceiver.

Este módulo actúa como interfaz entre el ESP32 y la red de domótica, y está compuesto por un microcontrolador PIC [11], en el que se implementa la biblioteca de comunicación, y un transceptor CAN MCP2551 [12], encargado de la adaptación de niveles eléctricos para la transmisión diferencial sobre el bus.

Como se puede apreciar en la figura 2.2, este módulo realiza la conversión de la red CAN, proveniente del sistema de domótica, a una interfaz de comunicación serial UART, compatible con el ESP32.

Esta solución permite desacoplar la gestión del protocolo de comunicación del procesamiento principal del sistema, lo que simplifica el desarrollo del firmware y facilita la integración del dispositivo en la red de domótica existente.

FIGURA 2.2. Funcionamiento del módulo transceiver.

2.2.3. Módulo Ethernet W5100

Para lograr la comunicación mediante el protocolo Modbus TCP, fue necesaria la utilización de un módulo Ethernet basado en el circuito W5100 [13].

El W5100 es un controlador Ethernet que permite dotar de conectividad de red a sistemas embebidos, gestionando internamente la pila de protocolos TCP/IP. La comunicación con el microcontrolador se realiza a través de una interfaz SPI, lo que facilita su integración en plataformas como el ESP32.

Mediante el uso de este módulo, el dispositivo desarrollado puede establecer conexiones sobre redes Ethernet, lo que permite la implementación del protocolo Modbus TCP para la comunicación con equipos del ecosistema Victron.

FIGURA 2.3. Módulo W5100.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

2.3. Herramientas de software

Para el desarrollo del software fue necesario definir tanto el entorno de desarrollo integrado como el lenguaje de programación a emplear. Asimismo, se evaluaron las herramientas necesarias para el desarrollo de la aplicación móvil.

2.3.1. Entorno de desarrollo Arduino

El microcontrolador ESP32 es compatible con el entorno de desarrollo Arduino [17], lo que permitió utilizar este IDE (*Integrated Development Environment*) como plataforma principal para el desarrollo del firmware. La elección de este entorno se basó en su facilidad de uso, amplia documentación y la disponibilidad de una gran cantidad de bibliotecas previamente desarrolladas.

En particular, el uso del IDE Arduino facilitó la implementación de los distintos protocolos de comunicación requeridos en el trabajo, tales como HTTP y Modbus TCP. Para este último, se emplearon bibliotecas específicas que fueron adaptadas para su funcionamiento sobre el módulo Ethernet W5100.

Adicionalmente, este entorno permite una rápida compilación y carga de programas sobre el dispositivo, así como el uso de herramientas de depuración básicas, lo que lo convierte en una opción adecuada para el desarrollo y prueba del sistema.

Para la implementación de la comunicación Modbus TCP, se utilizó una biblioteca desarrollada por Andre Sarmento Barbosa y Alexander Emelianov [18]. En conjunto con la biblioteca Ethernet provista por el entorno Arduino, esta herramienta permite la implementación del protocolo Modbus en distintas variantes, entre ellas Modbus TCP.

Para el desarrollo de la comunicación UART y del servidor interno en el ESP32, se emplearon las bibliotecas estándar provistas por defecto en el entorno de desarrollo.

2.3.2. Funcionamiento del ESP32

Para la programación del microcontrolador se optó por el lenguaje C++, debido a que es el lenguaje en el que se encuentran desarrolladas la mayoría de las bibliotecas disponibles en el entorno Arduino, lo que facilitó la integración de distintas funcionalidades requeridas en el trabajo.

Como metodología de desarrollo, se utilizó FreeRTOS [19], que se encuentra integrado en el ESP32. Este sistema operativo en tiempo real permite organizar el firmware mediante la ejecución de múltiples tareas, cuya planificación da lugar a un comportamiento concurrente a nivel lógico. Esto mejora la estructura del programa y facilita la gestión de distintas funcionalidades en paralelo.

Adicionalmente, FreeRTOS dispone de un planificador (*scheduler*) encargado de coordinar la ejecución de cada tarea. Este asigna el tiempo de procesamiento en función de su prioridad. Asimismo, proporciona mecanismos que permiten la comunicación y sincronización entre tareas, lo que facilita la correcta coordinación de las distintas funcionalidades implementadas en el sistema.

2.3. Herramientas de software 11

2.3. Herramientas de software

Para el desarrollo del software fue necesario definir tanto el entorno de desarrollo integrado como el lenguaje de programación a emplear. Asimismo, se evaluaron las herramientas necesarias para el desarrollo de la aplicación móvil.

2.3.1. Entorno de desarrollo Arduino

El microcontrolador ESP32 es compatible con el entorno de desarrollo Arduino [14], lo que permitió utilizar este IDE (*Integrated Development Environment*) como plataforma principal para el desarrollo del firmware. La elección de este entorno se basó en su facilidad de uso, amplia documentación y la disponibilidad de una gran cantidad de bibliotecas previamente desarrolladas.

En particular, el uso del IDE Arduino facilitó la implementación de los distintos protocolos de comunicación requeridos en el proyecto, tales como HTTP y Modbus TCP. Para este último, se emplearon bibliotecas específicas que fueron adaptadas para su funcionamiento sobre el módulo Ethernet W5100.

Adicionalmente, este entorno permite una rápida compilación y carga de programas sobre el dispositivo, así como el uso de herramientas de depuración básicas, lo que lo convierte en una opción adecuada para el desarrollo y prueba del sistema.

Para la implementación de la comunicación Modbus TCP, se utilizó una biblioteca desarrollada por Andre Sarmento Barbosa y Alexander Emelianov [15]. En conjunto con la biblioteca Ethernet provista por el entorno Arduino, esta herramienta permite la implementación del protocolo Modbus en distintas variantes, entre ellas Modbus TCP.

Para el desarrollo de la comunicación UART y del servidor interno en el ESP32, se emplearon las bibliotecas estándar provistas por defecto en el entorno de desarrollo.

2.3.2. Funcionamiento del ESP32

Para la programación del microcontrolador se optó por el lenguaje C++, debido a que es el lenguaje en el que se encuentran desarrolladas la mayoría de las bibliotecas disponibles en el entorno Arduino, lo que facilitó la integración de distintas funcionalidades requeridas en el proyecto.

Como metodología de desarrollo, se utilizó FreeRTOS [16], que se encuentra integrado en el ESP32. Este sistema operativo en tiempo real permite organizar el firmware mediante la ejecución de múltiples tareas, cuya planificación da lugar a un comportamiento concurrente a nivel lógico. Esto mejora la estructura del programa y facilita la gestión de distintas funcionalidades en paralelo.

Adicionalmente, FreeRTOS dispone de un planificador (*scheduler*) encargado de coordinar la ejecución de cada tarea. Este asigna el tiempo de procesamiento en función de su prioridad. Asimismo, proporciona mecanismos que permiten la comunicación y sincronización entre tareas, lo que facilita la correcta coordinación de las distintas funcionalidades implementadas en el sistema.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

2.3.3. Desarrollo de la aplicación móvil

Para el desarrollo de la aplicación móvil se optó por utilizar un conjunto de herramientas que trabajan de forma integrada. La figura 2.4 presenta un esquema de la relación entre las distintas tecnologías empleadas.

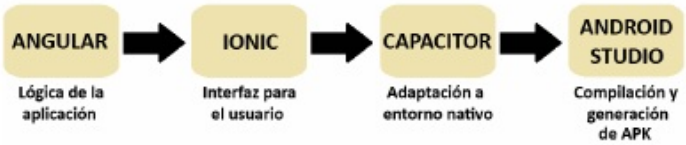

FIGURA 2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.

Se utilizó el framework Ionic [20] como base para el desarrollo de la aplicación, debido a que permite la creación de interfaces móviles multiplataforma a partir de tecnologías web.

Ionic se apoya en el framework Angular [21], que proporciona una estructura robusta para el desarrollo de interfaces de usuario dinámicas y organizadas en componentes. Angular utiliza lenguajes como TypeScript [22], HTML [23] y CSS [24], lo que facilita la construcción de aplicaciones modulares y escalables.

Para la adaptación de la aplicación a dispositivos móviles se utilizó Capacitor [25], herramienta que permite empaquetar la aplicación web como una aplicación nativa y facilita el acceso a funcionalidades propias del dispositivo.

Finalmente, se empleó Android Studio [26] para la compilación y generación del archivo APK (*Android Package Kit*), que constituye el paquete instalable de la aplicación en sistemas Android. Este archivo contiene todos los recursos necesarios para su ejecución, tales como el código compilado, las bibliotecas utilizadas y los archivos de configuración.

En este contexto, el firmware fue dividido en distintas tareas principales. En primer lugar, se implementó una tarea encargada de la gestión de la comunicación Wi-Fi, donde el ESP32 opera en modo *Access Point*. Este modo habilita la conexión de la aplicación móvil y permite al dispositivo funcionar como servidor HTTP para la recepción de solicitudes.

Por otra parte, se desarrollaron tareas dedicadas a la recepción de mensajes provenientes de la red CAN y a la atención de la comunicación Modbus TCP. Estas tareas se encargan de procesar la información recibida desde los distintos dispositivos conectados a la red de domótica.

Finalmente, se implementó una tarea central de procesamiento cuya función consiste en interpretar la información recibida desde la red CAN, desde la interfaz HTTP y desde el módulo de comunicación Modbus TCP, y en función de ello tomar las decisiones necesarias para el control del sistema.

2.3.3. Desarrollo de la aplicación móvil

Para el desarrollo de la aplicación móvil se optó por utilizar un conjunto de herramientas que trabajan de forma integrada. La figura 2.4 presenta un esquema de la relación entre las distintas tecnologías empleadas.

FIGURA 2.4. Esquema de integración entre Angular, Ionic, Capacitor y Android Studio.

Se utilizó el framework Ionic [17] como base para el desarrollo de la aplicación, debido a que permite la creación de interfaces móviles multiplataforma a partir de tecnologías web.

Ionic se apoya en el framework Angular [18], que proporciona una estructura robusta para el desarrollo de interfaces de usuario dinámicas y organizadas en componentes. Angular utiliza lenguajes como TypeScript, HTML y CSS, lo que facilita la construcción de aplicaciones modulares y escalables.

Para la adaptación de la aplicación a dispositivos móviles se utilizó Capacitor [19], herramienta que permite empaquetar la aplicación web como una aplicación nativa y facilita el acceso a funcionalidades propias del dispositivo.

Finalmente, se empleó Android Studio [20] para la compilación y generación del archivo APK (*Android Package Kit*), que constituye el paquete instalable de la aplicación en sistemas Android. Este archivo contiene todos los recursos necesarios para su ejecución, tales como el código compilado, las bibliotecas utilizadas y los archivos de configuración.

Ionic dispone de un amplio conjunto de bibliotecas que pueden ser utilizadas en el desarrollo de aplicaciones. Entre ellas, se encuentra una biblioteca destinada a la gestión de solicitudes HTTP.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Capítulo 3

Diseño e implementación

En este capítulo se explica cómo se diseñó el dispositivo, cómo está conformado el hardware y el software del equipo, y cómo se desarrolló la aplicación para el control del motorhome.

3.1. Desarrollo de hardware

Como se mencionó anteriormente, el ESP32 debe actuar como intermediario y vínculo entre las tres partes principales: módulo PRC10, Cerbo GX y aplicación móvil.

La figura 3.1 muestra una estructura del flujo o camino que debe seguir el ESP32 para llegar a cada una de estas partes.

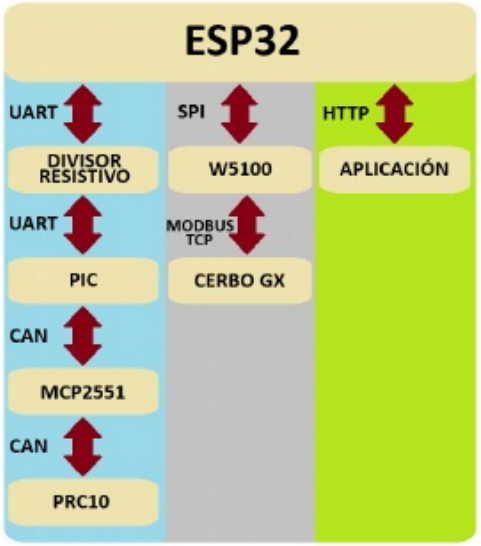

FIGURA 3.1. Trayecto que sigue el ESP32 para llegar a cada uno de los módulos.

En este contexto, se optó por utilizar dicha biblioteca como base para el desarrollo de funciones propias orientadas a la comunicación entre la aplicación y el ESP32.

Como se mencionó en la sección 2.1.3, la solicitud GET es utilizada para la realización de peticiones destinadas a la obtención de datos desde el servidor. A través de este mecanismo es posible consultar el estado de las distintas variables del sistema en el motorhome.

Por otra parte, mediante solicitudes POST es posible enviar datos desde la aplicación hacia el ESP32. Esto permite el control de las distintas variables del sistema y la ejecución de comandos sobre los dispositivos.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

La columna celeste representa la forma de comunicar el ESP32 con el módulo PRC10. Asimismo, las columnas gris y verde representan los caminos hacia el Cerbo GX y aplicación móvil respectivamente.

3.1.1. Comunicación con la red Enertik

Como se mencionó en la sección 2.1.1, la red de domótica de Enertik utiliza una biblioteca propia basada en el protocolo CAN. A su vez, todos los dispositivos de la red se conectan a través de un cableado de cuatro conductores: dos destinados a la alimentación de los equipos y dos destinados a la comunicación CAN (CAN H y CAN L).

Para el desarrollo de la comunicación del ESP32 mediante este protocolo, se consideró en primer lugar la adaptación de la biblioteca de Enertik dentro del firmware del dispositivo. Durante la etapa de investigación se identificaron bibliotecas disponibles que permiten al ESP32 implementar comunicación CAN a través de sus puertos GPIO.

Sin embargo, los intentos de establecer una comunicación básica entre el ESP32 y el módulo PRC10 no arrojaron resultados satisfactorios. Esto se debe a la complejidad de la biblioteca propia desarrollada por Enertik, cuyas condiciones y configuraciones no pudieron ser replicadas en el ESP32.

Ante esta situación, se analizaron alternativas disponibles dentro de los equipos comercializados por Enertik. En este contexto, se identificó el módulo transceiver (mostrado en la figura 3.2) utilizado en conjunto con la pantalla HMI de control.

Esta placa tiene como función realizar la conversión del protocolo CAN, proveniente de la red del motorhome, a una interfaz de comunicación UART, compatible con la pantalla.

FIGURA 3.2. Módulo transceiver comercial perteneciente a Enertik.

Basado en esta premisa, se decidió replicar y adaptar este módulo transceiver para que sea compatible con el dispositivo basado en ESP32. Este módulo se compone de un microcontrolador PIC y de un transceptor CAN MCP2551.

La figura 3.3 muestra un diagrama completo sobre como fue la implementación de este módulo transceiver.

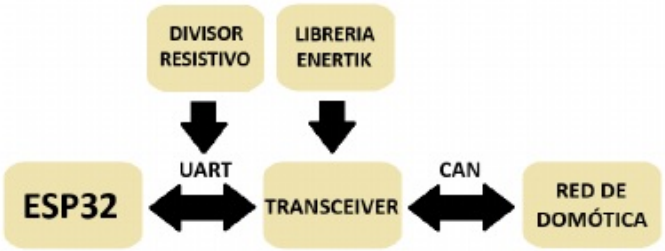

FIGURA 3.3. Diagrama de como se estable la comunicación CAN entre la red del motorhome y el ESP32.

El microcontrolador PIC dispone de la biblioteca de comunicación propia desarrollada por Enertik, lo que le permite interactuar de forma nativa con la red de domótica del motorhome. Por su parte, el transceptor MCP2551 se encarga de la adaptación de niveles eléctricos necesarios para la transmisión diferencial sobre el bus CAN.

El microcontrolador PIC funciona con una tensión de alimentación de 5 V, mientras que el ESP32 opera con 3,3 V. Esto implica que los niveles de tensión utilizados en los puertos UART de cada dispositivo son diferentes.

Esta diferencia generó la necesidad de adaptar los niveles de tensión en la comunicación UART, ya que una conexión directa podría dañar los puertos del ESP32 debido a su menor tolerancia de voltaje.

La solución adoptada fue la implementación de un divisor resistivo. La figura 3.4 muestra un diagrama del circuito utilizado, junto con el cálculo correspondiente a la tensión de salida.

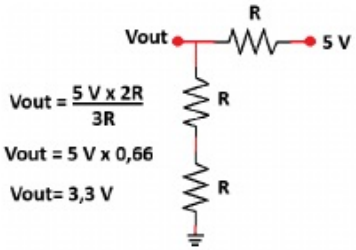

FIGURA 3.4. Diseño de divisor resistivo para adaptación UART.

Mediante el uso de tres resistencias de igual valor, se logró reducir la tensión de 5 V a 3,3 V en la línea de transmisión hacia el ESP32, lo que permitió una comunicación segura entre ambos dispositivos.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno lstlisting con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
 las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código 3.1:

```
#define MAX_SENSOR_NUMBER 3
#define MAX_ALARM_NUMBER  6
#define MAX_ACTUATOR_NUMBER 6

uint32_t sensorValue[MAX_SENSOR_NUMBER];
FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
state_t alarmState[MAX_ALARM_NUMBER];           //ON or OFF
state_t actuatorState[MAX_ACTUATOR_NUMBER];     //ON or OFF

void vControl() {

  initGlobalVariables();

  period = 500 ms;

  while(1) {

    ticks = xTaskGetTickCount();

    updateSensors();

    updateAlarms();

    controlActuators();

    vTaskDelayUntil(&ticks, period);
  }
```

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Este divisor fue implementado en la conexión correspondiente a la línea de transmisión UART (Tx) del microcontrolador PIC hacia la línea de recepción UART (Rx) del ESP32. En el sentido contrario no fue necesaria esta adaptación, ya que el nivel lógico de 3,3 V proveniente del ESP32 es reconocido correctamente por el PIC.

De esta manera, el módulo transceiver actúa como un intermediario que realiza la conversión entre la red CAN y una interfaz UART. Esto permite que el ESP32 pueda enviar y recibir información de la red de domótica sin necesidad de implementar directamente la complejidad del protocolo CAN utilizado por Enertik.

3.1.2. Comunicación con la red Victron

La comunicación con la red Victron se realizó mediante el protocolo Modbus TCP. Desde el punto de vista del hardware, el microcontrolador ESP32 no dispone de una interfaz Ethernet integrada, por lo que fue necesario incorporar un módulo externo que permita este tipo de conectividad.

Durante la etapa de investigación se analizaron distintas alternativas para implementar la comunicación Ethernet. Como resultado, se optó por utilizar el módulo W5100, debido a su amplia disponibilidad en el mercado, su bajo costo y su compatibilidad con entornos de desarrollo basados en Arduino.

Adicionalmente, el IDE Arduino dispone de bibliotecas específicas para la comunicación Ethernet que resultan compatibles con este módulo, lo que facilitó su integración dentro del sistema. Estas características hicieron del W5100 una opción adecuada para la implementación de la comunicación Modbus TCP con los equipos del ecosistema Victron.

Otra ventaja de este componente es que utiliza el protocolo de comunicación SPI. El ESP32 dispone de periféricos compatibles con este protocolo, lo que facilitó su integración dentro del sistema.

Más allá del conexionado SPI (pines MOSI, MISO, SCK y CS), la conexión entre el ESP32 y el W5100 no requirió hardware adicional.

Este módulo funciona con una tensión de alimentación de 5 V, lo que lo vuelve compatible con el diseño implementado, ya que se aprovecha la misma fuente utilizada para energizar el microcontrolador PIC del módulo transceiver.

La conexión Ethernet que parte desde el módulo W5100 se realiza a través de un conector RJ45. Este conector permite la utilización de cables de red estándar, compuestos por pares trenzados de conductores, diseñados para la transmisión de datos a alta velocidad.

Estos cables emplean una transmisión diferencial sobre cada par, lo que reduce la susceptibilidad a interferencias electromagnéticas y mejora la integridad de la señal. De esta manera, se garantiza una comunicación confiable entre el dispositivo desarrollado y la red del sistema Victron.

Mediante este tipo de conexión, el módulo W5100 puede integrarse directamente a la red Ethernet donde se encuentra el módulo Cerbo GX, permitiendo el intercambio de datos necesario para la implementación del protocolo Modbus TCP.

16 Capítulo 3. Diseño e implementación

```
}
```

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

Para lograr la comunicación, resta definir qué dispositivo cumplirá el rol de maestro y cuál el de esclavo. Esta definición se abordará en la sección de firmware, donde se detallará su implementación.

3.1.3. Prototipo experimental

Para realizar los primeros ensayos, se decidió montar un primer prototipo del equipo. Para ello, se utilizó como base un PCB perforado de tipo experimental, sobre el que se soldaron diversos conectores, cableados y componentes de forma manual.

La figura 3.5 muestra el resultado del montaje del prototipo utilizado para los ensayos iniciales.

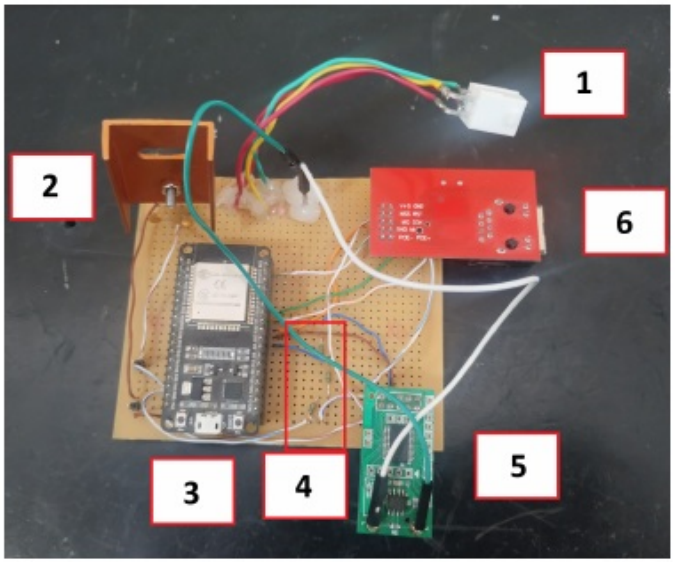

FIGURA 3.5. Prototipo experimental del equipo.

Como se puede apreciar, el prototipo cuenta con cada uno de los módulos explicados en secciones anteriores, donde:

- Componente 1: conector de cuatro hilos. Este conector es el estándar empleado en la red de domótica de Enertik. A través de él se energizan todos los componentes del PCB experimental y se accede a la comunicación CAN de la red.
- Componente 2: regulador de tensión 7805 [27]. Este dispositivo se encarga de suministrar una tensión estable de 5 V. Dado que la tensión proveniente del motorhome es de 12 V, fue necesaria su utilización para adecuar los niveles de alimentación a los requeridos por los módulos.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

- Componente 3: módulo ESP32. Es el microcontrolador principal de la placa y contiene el firmware de control del sistema. En este prototipo se alimentó a través del regulador 7805, ya que la placa de desarrollo dispone internamente de reguladores que reducen la tensión a 3,3 V. Para el desarrollo final fue necesaria la incorporación de un regulador adicional dedicado al ESP32.
- Componente 4: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceiver.
- Componente 5: módulo transceiver. Se trata de una placa comercial provista por Enertik. En este prototipo se decidió incorporar un conector en el PCB experimental para facilitar su montaje y desmontaje de forma segura.
- Componente 6: módulo W5100. De manera similar al módulo transceiver, se utilizó un conector compatible que permite su montaje y desmontaje de forma sencilla y segura.

Este prototipo permitió validar el funcionamiento general del sistema y realizar pruebas iniciales de integración entre los distintos módulos de hardware.

3.1.4. Desarrollo de PCB final y listado de componentes

El diseño del PCB final se desarrolló mediante el software Altium Designer [28]. Este software es ampliamente utilizado en la industria, ya que permite crear esquemáticos de circuitos de forma sencilla. Estos esquemáticos sirven como base para el diseño de placas de circuito impreso.

Para la creación de este PCB, se tomó como referencia el conexionado utilizado en el prototipo experimental.

Dado que se espera que este desarrollo se convierta en un producto comercial, fue necesario realizar modificaciones en la selección de componentes durante el diseño de este prototipo.

La primera diferencia se encuentra en el microcontrolador ESP32. Para el prototipo experimental y los ensayos iniciales se utilizó una placa de desarrollo comercial basada en este dispositivo.

En el diseño del PCB final, se optó por reemplazar dicha placa por el chip del microcontrolador en sí. Esta decisión permitió descartar los componentes adicionales presentes en la placa de desarrollo. De esta forma, se eliminó hardware innecesario, se redujeron costos y se optimizó el espacio disponible en el PCB.

Por otra parte, fue necesaria la incorporación de un regulador de tensión AP63203 [29]. Este dispositivo permite adaptar el nivel de tensión de 12 V proveniente de la red del motorhome a un valor de 3,3 V, compatible con el chip ESP32.

En el caso del módulo transceiver, dado que se conocía completamente su diseño, se optó por integrarlo directamente en el PCB. Para ello, se incorporaron el microcontrolador PIC18F26K60 y el transceptor MCP2551 en la placa. Esta decisión permitió optimizar el espacio disponible y reducir costos al unificar todos los componentes en una única placa.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

En cuanto al módulo W5100, se optó por utilizar la versión comercial disponible en el mercado. Inicialmente se evaluó realizar un proceso de ingeniería inversa para analizar su diseño interno, pero se descartó esta alternativa debido al tiempo requerido.

Como consecuencia, en el PCB se reservó un espacio para la colocación de un conector compatible con el módulo W5100, lo que permite su montaje manual de forma sencilla.

En la figura 3.6 se puede observar el diseño final del PCB.

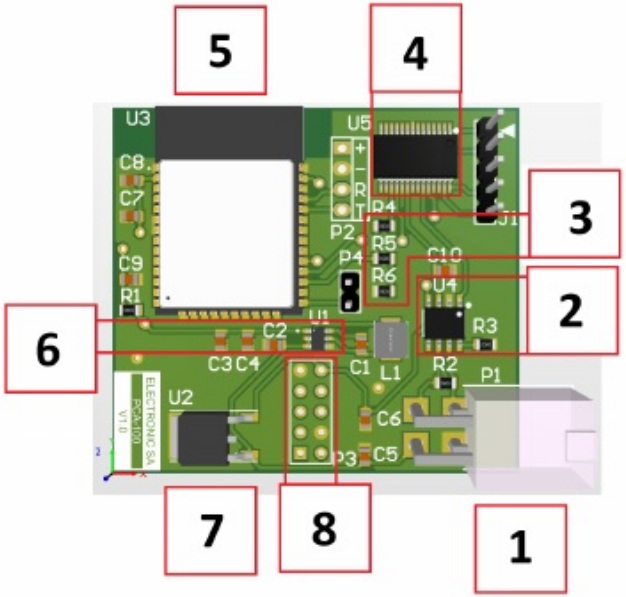

FIGURA 3.6. Diseño final para PCB del equipo.

Como se puede apreciar, el PCB esta compuesto por los siguientes componentes:

- Componente 1: conector principal. Terminal sobre que se conecta el cableado de cuatro hilos proveniente del motorhome.
- Componente 2: MCP2551. Forma parte del módulo transceiver, encargado de adaptar los niveles de tensión de la linea CAN.
- Componente 3: divisor resistivo. Corresponde al arreglo de resistencias encargado de adaptar los niveles de tensión y permitir una comunicación UART segura entre el ESP32 y el módulo transceiver.
- Componente 4: PIC18F26K80. Microcontrolador que contiene la biblioteca desarrollada por Enertik para la comunicación entre equipos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se puedo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

- Componente 5: módulo ESP32. Microcontrolador que contiene el firmware principal del equipo.
- Componente 6: regulador de tensión AP63203. Fuente de tensión de 3,3 V encargado alimentar el ESP32.
- Componente 7: regulador de tensión 7805. Componente responsable de suministrar una tensión estable de 5 V para alimentar el PIC y el W5100.
- Componente 8: conector de diez pines. Conector compatible con el módulo W5100. En este lugar se montará dicho componente.

Para el proceso de fabricación, fue necesaria la contratación de servicios de un fabricante especializado en este campo. Para ello, se generaron los denominados archivos Gerber [30].

Los archivos Gerber constituyen el estándar utilizado en la industria para describir las distintas capas de una placa de circuito impreso. Cada archivo representa una capa específica del PCB, como las pistas de cobre, la máscara de soldadura, la serigrafía o el plano de taladros. A partir de esta información, el fabricante puede interpretar el diseño y llevar a cabo la fabricación física de la placa con precisión.

A modo de resumen, la tabla 3.1 lista los principales componentes empleados para la fabricación del PCB.

TABLA 3.1. Componentes utilizados en el PCB final.

Componente	Descripción
ESP32	Microcontrolador principal del equipo.
PIC18F26K80	Encargado de la comunicación CAN.
MCP2551	Encargado de adaptación de niveles en la comunicación CAN.
7805	Regulador de tensión de 5 V.
AP63203	Regulador de tensión de 3,3 V.
Resistencias	Arreglo para el divisor de tensión.
W5100	Módulo para comunicación Ethernet.

3.2. Desarrollo de firmware

Como se mencionó en la sección 2.3.2, se optó por utilizar FreeRTOS como base para el desarrollo del firmware.

FreeRTOS permite estructurar el firmware mediante la ejecución de múltiples tareas. La planificación de estas tareas genera un comportamiento concurrente a nivel lógico, lo que mejora la organización del sistema y facilita la gestión de sus funcionalidades.

En este contexto, en el presente trabajo se implementaron seis tareas principales, ilustradas en la figura 3.7, donde se aprecian las relaciones existentes entre ellas y con los módulos externos.

DiffPDF • /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/TI_Rodriguez_Matias_Nahuel_V3.pdf vs. /home/usuario/Escritorio/Plantilla-para-memoria/Entregas/

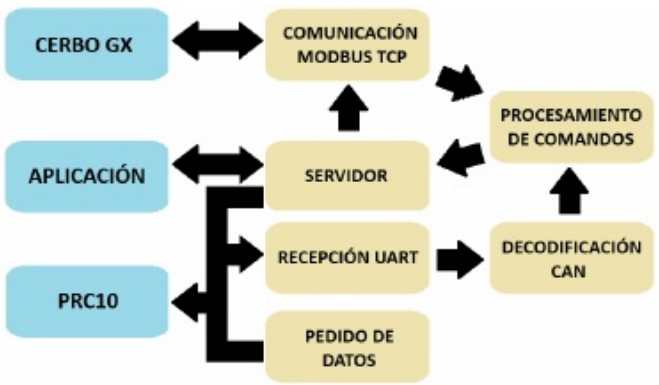

FIGURA 3.7. Relación entre las distintas tareas del firmware y su comunicación con el exterior.

3.2.1. Tarea: recepción UART

Esta tarea se encarga de escuchar constantemente el puerto UART del ESP32. Cada vez que se detecta una señal en sus puertos, la tarea almacena la información recibida en un buffer de tipo circular [31].

Un buffer circular, también conocido como *ring buffer*, es una estructura de datos que utiliza una región de memoria de tamaño fijo y organiza los datos de forma continua. Cuando se alcanza el final del buffer, la escritura continúa desde el inicio, lo que permite reutilizar la memoria de manera eficiente sin necesidad de realizar desplazamientos de datos.

Se optó por este tipo de estructura porque resulta especialmente adecuado para la recepción de datos en tiempo real, ya que permite desacoplar la velocidad de llegada de la información respecto de su procesamiento. De esta forma, se evita la pérdida de datos ante ráfagas de información y se garantiza un manejo más robusto de la comunicación UART.

3.2.2. Tarea: decodificación de comando CAN

Esta tarea se encarga de decodificar los comandos recibidos desde la red CAN. Para ello, realiza la lectura de los datos almacenados en el buffer circular.

La información transmitida en la red CAN sigue un formato predefinido. El primer byte corresponde a una marca de inicio, que permite identificar el comienzo de cada trama. Por otro lado, el último byte corresponde a un valor de control CRC (*Cyclic Redundancy Check*), utilizado para verificar la integridad de los datos recibidos mediante un cálculo basado en el contenido del mensaje.

Dentro de la trama de datos se incluyen los campos de dirección de origen, que identifica el dispositivo emisor, dirección de destino, que indica el dispositivo receptor, un campo de comando asociado a la acción a ejecutar y un conjunto de bytes de datos.

Bibliografía

[1] Enertik Argentina. Última consulta: 2025-03-14. URL: https://enertik.com/ar/.

[2] Victron Energy. Última consulta: 2025-03-14. URL: https://www.victronenergy.com.es/.

[3] Enertik Argentina. *PRC10: manual para el usuario.*

[4] Espressif. Última consulta: 2025-03-21. URL: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/get-started/index.html.

[5] *Cerbo GX*. Última consulta: 2025-03-16. URL: https://www.victronenergy.com.es/communication-centres/cerbo-gx.

[6] *CAN Bus Explained - A Simple Intro [2025]*. Última consulta: 2025-03-16. URL: https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial.

[7] Manual de GX - Modbus TCP. Última consulta: 2025-03-15. URL: https://www.victronenergy.com/live/ccgx:modbustcp_faq.

[8] *Modbus: Qué es y cómo funciona*. Última consulta: 2025-03-19. URL: https://www.cursosaula21.com/modbus-que-es-y-como-funciona/.

[9] *TCP/IP Overview*. Última consulta: 2025-03-25. URL: https://www.cisco.com/c/en/us/support/docs/ip/routing-information-protocol-rip/13769-5.html.

[10] *Overview of HTTP*. Última consulta: 2025-03-19. URL: https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview.

[11] *PIC18F66K80 FAMILY*. Última consulta: 2025-03-19. URL: https://ww1.microchip.com/downloads/en/DeviceDoc/PIC18F66K80%20FAMILY%20Enhanced%20Flash%20MCU%20with%20ECAN%20XLP%20Technology%2030009977G.pdf.

[12] *MCP2551*. Última consulta: 2025-03-19. URL: https://ww1.microchip.com/downloads/en/devicedoc/20001667g.pdf.

[13] *W5100*. Última consulta: 2025-03-19. URL: https://cdn.sparkfun.com/assets/e/2/f/c/2/W5100_Datasheet_v1_1_6.pdf.

[14] *Arduino*. Última consulta: 2025-03-25. URL: https://docs.arduino.cc/software/ide-v1/tutorials/PortableIDE/.

[15] *Github: modbus-esp8266*. Última consulta: 2025-03-25. URL: https://github.com/emelianov/modbus-esp8266.

[16] *FreeRTOS*. Última consulta: 2025-03-19. URL: https://www.freertos.org/.

[17] *Introduction to Ionic*. Última consulta: 2025-03-19. URL: https://ionicframework.com/docs.

[18] *Arduino*. Última consulta: 2025-03-25. URL: https://angular.dev/.

[19] *Capacitor web*. Última consulta: 2025-03-25. URL: https://capacitorjs.com/.

[20] *Android Studio*. Última consulta: 2025-03-25. URL: https://developer.android.com/studio?hl=es-419.